

JS 호이스팅과 클로저 개념정리.

1. 호이스팅 (Hoisting)

호이스팅은 변수와 함수의 선언문이 코드의 최상단으로 끌어올려진 것처럼 동작하는 현상을 말합니다. 자바스크립트 엔진이 코드를 실행하기 전, 컴파일 단계에서 메모리에 변수와 함수를 먼저 등록하기 때문에 발생합니다. [1, 2, 3, 4]

① 변수 호이스팅 (`var` vs `let`, `const`)

모든 변수는 호이스팅되지만, 선언 키워드에 따라 동작이 다릅니다.

- `var`: 선언과 동시에 `undefined` 로 초기화됩니다. 선언문 이전에 호출해도 에러가 나지 않고 `undefined` 를 반환합니다.
- `let`, `const`: 선언은 되지만 초기화되지 않습니다. 선언문 이전에 접근하면 TDZ(Temporal Dead Zone, 일시적 사각지대)에 걸려 `ReferenceError` 가 발생합니다. [1, 2, 3, 4, 5]

javascript

```
console.log(nameVar);// 출력: undefined (에러 안 남)varnameVar ="joell";  
console.log(nameLet);// 에러: ReferenceError (TDZ 걸림)letnameLet ="joell";
```

코드를 사용할 때는 주의가 필요합니다.

② 함수 호이스팅

함수를 선언하는 방식에 따라서도 다르게 동작합니다.

- 함수 선언문 (`function func() {}`): 함수 전체가 호이스팅되므로, 선언 전에도 함수를 정상적으로 호출할 수 있습니다.
- 함수 표현식 (`var func = function() {}`): 변수 호이스팅 규칙을 따릅니다. `var` 로 만들면 `undefined` 가 되어 호출 시 `TypeError` 가 나고, `let/const` 로 만들면 `ReferenceError` 가 납니다.

javascript

```
sayHello();// 출력: "안녕하세요!" (정상 작동)functionsayHello() {  
  console.log("안녕하세요!");  
}  
  
sayBye();// 에러: TypeError (sayBye는 현재 undefined이기 때문에 함수로 호출 불가)varsayBye =
```

```
function() {
  console.log("안녕하하세요!");
};
```

코드를 사용할 때는 주의가 필요합니다.

2. 클로저 (Closure)

클로저는 외부 함수가 종료되어 반환된 후에도, 내부 함수가 외부 함수의 지역 변수에 계속 접근할 수 있는 현상과 그 환경을 의미합니다. 자바스크립트는 함수가 선언된 주변 환경(렉시컬 스코프)을 기억하기 때문에 가능합니다. [1, 2, 3]

① 클로저의 기본 구조

아래 예제에서 `outer` 함수는 실행이 끝났지만, 내부의 `inner` 함수가 외부의 `outerVar` 를 기억하고 있어서 호출할 때마다 정상적으로 값을 가져옵니다. [1, 2]

javascript

```
function outer() {
  let outerVar = "외부 변수";

  return function inner() {
    console.log(outerVar); // 외부 함수의 변수에 접근
  };
}

const myClosure = outer(); // outer 함수는 여기서 실행이 끝남
myClosure(); // 출력: "외부 변수" (끝난 함수의 변수를 여전히 기억함)
```

코드를 사용할 때는 주의가 필요합니다.

② 클로저를 왜 쓸까?

가장 대표적인 용도는 **데이터 은닉(Private 변수 구현)**과 **상태 유지**입니다. 외부에서 직접 변수를 수정하지 못하도록 보호할 수 있습니다. [1, 2, 3]

javascript

```
function createCounter() {
  let count = 0; // 외부에서 직접 접근 불가 (Private 변수 효과)

  return {
    increase: function() { count++; return count; },
    decrease: function() { count--; return count; }
  };
}

const counter = createCounter();
```

```
console.log(counter.increase()); // 출력: 1
console.log(counter.increase()); // 출력: 2
console.log(counter.count);      // 출력: undefined (직접 접근 불가로 데이터 보호)
```

코드를 사용할 때는 주의가 필요합니다.

한 줄 요약

- **호이스팅**: 자바스크립트 엔진이 실행 전에 선언문들을 미리 메모리에 올려두어, 선언 전에도 존재하는 것처럼 작동하는 현상.
- **클로저**: 이미 생명 주기가 끝난 외부 함수의 변수를 내부 함수가 마치 "자식"처럼 쥐고서 계속 기억하고 활용하는 현상.

1. 호이스팅 (Hoisting)의 깊은 원리

호이스팅을 직역하면 "끌어올리다"이지만, **실제로 코드가 위로 이동하는 것은 아닙니다**. 브라우저가 코드를 실행하기 전, 내부적으로 거치는 **2가지 단계** 때문에 일어나는 현상입니다.

자바스크립트 엔진의 2단계 동작

1. **생성 단계 (Creation Phase)**: 코드를 쭉 훑으면서 변수 선언문과 함수 선언문만 골라내어 메모리 공간을 미리 할당합니다. (이때 호이스팅이 일어남)
2. **실행 단계 (Execution Phase)**: 위에서부터 한 줄씩 코드를 실행하며 변수에 실제 값을 대입(=)하고 함수를 호출합니다.

변수 키워드별 선언과 초기화 과정

자바스크립트 변수는 선언 과정이 **선언 → 초기화 → 할당** 3단계로 나뉩니다.

```
[ var 의 경우 ]
생성 단계 : [ 선언 + undefined로 초기화 ] <-- 선언하자마자 undefined가 방에 들어감
실행 단계 : [ 값 할당 ]

[ let / const 의 경우 ]
생성 단계 : [ 선언 ] <-- 메모리에 이름만 등록 (초기화 안 됨!)
실행 단계 : [ 초기화 ] → [ 값 할당 ] <-- 실제 코드 선언문에 도달해야 초기화됨
```

- **var** 는 생성 단계에서 이름표를 붙이면서 동시에 내부 값을 **undefined** 로 채워둡니다. 그래서 선언 전 호출해도 에러가 안 납니다.
- **let**, **const** 는 생성 단계에서 이름만 등록하고 내부 공간은 비워둡니다. 실제 코드에서 **let x;** 라인을 만나기 전까지는 알맹이가 없기 때문에 접근할 수 없으며, 이 이름만 있고 사용할 수 없는 상태의 구역을 **TDZ(일시적 사각지대)**라고 부릅니다.

2. 클로저 (Closure)의 깊은 원리

클로저를 이해하려면 자바스크립트 엔진이 방을 만드는 기준인 **실행 컨텍스트**와 **렉시컬 스코프(Lexical Scope)**를 알아야 합니다.

⚙️ 렉시컬 스코프 (Lexical Scope)

자바스크립트는 함수를 "어디서 실행했는지"가 아니라 "어디서 선언(정의)했는지"에 따라 상위 스코프(부모 방)를 결정합니다. 이 뼈대는 코드가 작성될 때 이미 정적으로 결정되며, 절대 변하지 않습니다.

🔍 메모리 관점에서의 클로저 발생 원인

원래 일반적인 프로그래밍 언어에서는 함수 실행이 끝나면 그 함수가 쓰던 메모리 공간(지역 변수들)을 자동으로 파괴(Garbage Collection)합니다. 하지만 자바스크립트는 다릅니다.

javascript

```
function outer() {
  let money = 10000; // outer 변수
  return function inner() {
    console.log(money); // inner가 outer의 money를 참조함
  };
}
const myClosure = outer(); // outer() 실행 컨텍스트는 사라짐. 하지만...
```

코드를 사용할 때는 주의가 필요합니다.

1. `outer()` 함수가 실행되면 메모리에 `money = 10000` 이 담긴 `outer 공간` 이 만들어집니다.
2. `outer()` 가 종료되면서 내부의 `inner` 함수를 밖으로 던져주고(`return`), 자신은 실행 컨텍스트 스택에서 제거됩니다.
3. 이때, 반환된 `inner` 함수는 태어날 때부터 부모인 `outer 공간` 을 기억하는 비밀 통로(`[[Environment]]` 라는 내부 프로퍼티)를 가지고 있습니다.
4. 자바스크립트 엔진(가비지 컬렉터)은 "누군가 참조하고 있는 메모리 공간은 절대 지우지 않는다"는 규칙이 있습니다. `myClosure` 변수가 `inner` 함수를 쥐고 있고, `inner` 함수가 `outer 공간` 을 붙잡고 있기 때문에 `outer 공간` 은 파괴되지 않고 메모리에 계속 살아남아 클로저가 형성됩니다.

핵심 요약

- **호이스팅이란?** 자바스크립트 엔진이 소스코드를 평가하는 **생성 단계**에서 변수와 함수 선언문을 메모리에 선언(등록)하기 때문에, 코드 순서상 선언문보다 위에서 변수나 함수에 접근할 수 있게 되는 현상입니다.
- **클로저란?** 내부 함수가 유효한 상태에서 외부 함수가 종료되어도, 내부 함수가 자신이 선언되었을 때의 환경(렉시컬 스코프)을 기억하여 **외부 함수의 지역 변수를 계속해서 참조**

할 수 있는 함수와 그 환경의 조합입니다.